

CS 4347.003 - Database Systems

Professor Jalal Omer

LabBench

Final Project Report

Group 18

Uzayr Chaudhry

Ugonna Anyalemechi

Yasar Labib

Zuhaib Buchh

May 3, 2026

Table of Contents

I. Cover Page	1
II. Table of Contents	2
III. Introduction	4
IV. Requirements Analysis	6
IV.1 User Needs	6
IV.2 Functional Requirements	6
IV.3 Nonfunctional Requirements	8
IV.4 Assumptions	9
IV.5 Constraints	9
IV.6 Requirement-to-Implementation Summary	10
V. Logical Database Design	11
V.1 ER Diagram	11
V.2 Entity and Relationship Notes	11
V.3 Participation and (Min, Max) Constraints	12
V.4 Relational Database Schema	13
V.5 Relational Schema Diagram	16
V.6 Referential Integrity Constraints	16
V.7 Mapping from ER/EER Diagram to Relational Schema	17
V.8 Normalization	17
V.8.1 Normalization Criteria Used	17
V.8.2 Initial Schema to Final Schema	18
V.8.3 Users	18
V.8.4 Workspaces	19
V.8.5 WorkspaceMembers	19
V.8.6 Projects	19
V.8.7 Datasets	20
V.8.8 DatasetVersions	20
V.8.9 Models	20
V.8.10 Runs	21
V.8.11 RunParams	21
V.8.12 RunMetrics	22
V.8.13 ModelRegistry	22
V.8.14 AuditLog	22
VI. Database Implementation	23

VI.1 Overview	23
VI.2 create.sql	23
VI.3 load.sql	28
VII. Application and User Interfaces	31
VII.1 Overview	31
VII.2 Interfaces and User Interactions	31
VII.3 Database Connectivity	35
VIII. Results	35
VIII.1 Sample Select Query	35
VIII.2 Sample Update Query	36
VIII.3 Sample Delete Query	36
VIII.4 Sample View Query	37
IX. Conclusion and Future Work	38
IX.1 Conclusion	38
IX.2 Limitations	39
IX.3 Future Work	40
X. References and Appendix	41
X.1 References	41
X.2 Appendix A: Full SQL Code	41
X.3 Appendix B: Interface Screenshots	48

III. Introduction

LabBench is a web-based application backed by a database that helps machine learning teams organize, track, compare, and review the results of their experimental work with machine learning models. Machine learning researchers typically conduct numerous experiments within a common project, using different datasets, versions of the same dataset, model architectures, hyperparameter configurations, code versions, and evaluation metrics. In many cases, machine learning researchers record and track this information in spreadsheets, notebooks, or informal notes; this lack of structure makes it difficult to reproduce prior work, compare experiments consistently, eliminate duplicate or inconsistent entries, and maintain a reliable history of model development.

LabBench provides a solution for machine learning research teams by serving as a management repository for machine learning experiment metadata and results. A typical LabBench user may be a student working as part of a team on an ML project, a university researcher, or an employee at a small company or organization focused on engineering machine learning applications. LabBench provides these users with a single structured system for storing projects, datasets, dataset versions, models, training runs, run parameters, run metrics, and approved model versions, using a relational database design rather than a flat spreadsheet.

The purpose of LabBench is to provide a structured database system for tracking the full workflow from workspace-level collaboration down to individual model runs. The core workflow is:

Workspace → Project → Model → Run

A workspace is the collaboration environment for a team. Projects organize related machine learning work. Models represent experiment families, model architectures, or machine learning approaches within a project. Runs represent individual training or evaluation attempts using a specific model and dataset version. Each run can have many parameters, such as learning rate or batch size, and many metrics, such as accuracy, loss, precision, recall, or F1-score. The system also includes a model registry that maintains a list of approved model versions and an audit log that records the history of significant user actions.

The final implementation includes LabBench's relational database and web application interface. The database schema includes tables for Users, Workspaces, WorkspaceMembers, Projects, Datasets, DatasetVersions, Models, Runs, RunParams, RunMetrics, ModelRegistry, and AuditLog. The web application allows users to log in to LabBench, browse and view project details, manage workspace members, view datasets and dataset versions, log runs, add run

parameters and metrics, update run information, compare runs, view model versions in the model registry, and review audit logs. The primary focus of the LabBench system is to store experimental metadata and experiment results. The system does not store raw training data, such as image files, text corpora, large datasets, notebooks, or trained model artifact files.

The technologies used for the LabBench application are SQL, MySQL, PHP, HTML, and CSS. SQL scripts create the schema and load sample CSV data. PHP powers server-side application logic, database connectivity, session handling, form processing, validation, and CRUD operations. HTML and CSS are used to build the user interface. The application uses PDO to connect PHP to MySQL and to execute database operations. The final local demo also included a SQL injection demonstration for the required group assignment.

LabBench highlights how relational database principles can facilitate real machine learning experimentation by solving an experiment-tracking problem. The project includes conceptual database design, ER/EER modeling, relational schema mapping, normalization, SQL implementation, data loading, a working PHP/MySQL web interface, CRUD operations, query results, validation, run comparison, audit logging, and a security-focused SQL injection prevention demonstration.

IV. Requirements Analysis

LabBench was designed to support teams that need to manage machine learning experiment records in a consistent, searchable, and relationally structured way. At a high level, the system must allow authenticated users to access shared workspaces, manage projects, track datasets and dataset versions, manage models, log runs, track run parameters and metrics, compare the results of their completed experiments, register approved versions of the models, and perform an audit of all activity.

IV.1 User Needs

The primary users of LabBench include machine learning teams with students, research labs, and small groups that are developing machine learning products. These users generally require a centralized system that maintains a complete history of experiments and supports comparisons. In a realistic machine learning workflow, team members will experiment with multiple architectures, hyperparameter variations, multiple dataset versions, and evaluate results along numerous metrics. The system must maintain a record of these historically significant pieces of information so that results from prior dates can be referenced and repeated from memory.

IV.2 Functional Requirements

The final LabBench application must support the following functional requirements:

1. User authentication

- Users must be able to log in with an email and password.
- The system must maintain a session after login.
- Protected pages must not be accessible to users who are not logged in.

2. Workspace and user management

- The system must support multiple users.
- Users may belong to workspaces through workspace membership records.
- Workspace membership must identify each user's role, such as admin or member.
- Admin-only areas should be protected from non-admin users.

3. Project management

- Users must be able to view projects within a workspace.
- Users must be able to create, view, update, and delete projects when allowed.
- Each project must belong to one workspace.
- Each project must keep a reference to the user who created it.

4. Dataset and dataset version tracking

- The system must store datasets associated with projects.

- Each dataset must have one or more dataset versions.
- Dataset versions must include a version tag and metadata such as row count and schema hash when available.
- Runs must reference a specific dataset version so that experiments are reproducible.

5. Model tracking

- The system must store models within projects.
- A model represents an experiment family or machine learning approach.
- Runs must reference a valid model.

6. Run management

- Users must be able to create a run for a selected model and dataset version.
- Runs must store status, code version tag, timestamps, notes, and creator information.
- Users must be able to view, update, and delete runs when allowed.
- Run deletion must respect referential integrity. For example, a run used in the model registry should not be deleted if the database restricts that action.
- Runs must be searchable and filterable by relevant fields such as project, status, code version tag, or notes.

7. Run parameter management

- Each run may have multiple parameters.
- Parameters must be stored separately from the main run record.
- A parameter must include a parameter key and value.
- Users must be able to add and delete parameters for a run.

8. Run metric management

- Each run may have multiple metrics.
- Metrics must be stored separately from the main run record.
- A metric must include a metric key, numeric value, step, and recorded timestamp.
- Users must be able to add and delete metrics for a run.

9. Run comparison

- Users must be able to select two runs and compare them side by side.
- The comparison should show run summary information and metric values.
- Metrics should be aligned by metric key and step where possible.

10. Model registry

- The system must support registry entries for approved or selected model versions.
- A registry entry must reference the model, the source run, and the approving user.

- A source run should remain protected if it is referenced by the model registry, preserving reproducibility history.

11. Audit logging

- The system should record important actions such as create, update, delete, or approval operations.
- Audit log records should identify the workspace, acting user, action type, entity type, entity ID, and timestamp.

12. SQL injection demonstration

- The system must include a demonstration form that performs a vulnerable SELECT query.
- The vulnerable version must show how SQL injection can retrieve rows even when the input is incomplete or inaccurate.
- The system must also include a prepared-statement version of the same SELECT routine to show how the attack is prevented.

IV.3 Nonfunctional Requirements

LabBench must also satisfy several nonfunctional requirements:

1. Data integrity

- Primary keys must uniquely identify records.
- Foreign keys must enforce valid relationships between tables.
- Cascading deletes should be used only for dependent detail records that have no meaning without their parent.
- Restrictive delete behavior should be used for major historical records, such as models, runs, registry entries, and audit logs, where preserving history matters.

2. Usability

- Pages should be organized with clear navigation.
- Forms should be understandable and connected to the correct database actions.
- Error messages should be readable and should not expose raw database internals during normal use.
- Query results should be displayed in tables or structured views.

3. Security

- Users should authenticate before accessing protected pages.
- Admin-only pages should not be available to regular users.
- Normal application pages should use safe database access patterns.
- The intentionally vulnerable SQL injection logic should be isolated to the dedicated assignment demonstration page.

- Prepared statements should be used to separate SQL code from user input.

4. Reliability

- The application should handle invalid or missing input without crashing.
- The database should prevent orphaned records.
- The application should continue to function after normal create, update, and delete operations.
- Database setup should be repeatable through create.sql, load.sql, and CSV files.

5. Maintainability

- Shared logic such as database connection, session protection, helper functions, navigation, and audit logging should be centralized where possible.
- PHP pages should use the shared PDO database connection.
- The schema should remain understandable and aligned with the ER model and relational mapping.

IV.4 Assumptions

The project is based on the following assumptions:

- The application is intended for a course project demo and runs in a local development environment.
- The database is named LabBench.
- Sample data is loaded from CSV files using load.sql.
- Users are seeded for demo purposes.
- The system stores machine learning experiment metadata, not raw datasets or trained model files.
- A run belongs to exactly one model and uses exactly one dataset version.
- A run can have many parameters and many metrics.
- A workspace can contain many projects.
- A project can contain many datasets and many models.
- A model can have many runs.
- A dataset can have many versions.
- A model registry entry is based on a source run and should preserve that run's history.

IV.5 Constraints

The project has several implementation and course constraints:

1. The database must be implemented using MySQL.
2. The website must communicate with the database and perform CRUD actions.
3. The final demo must show working forms, query results, updates, deletes, validation, and database connectivity.

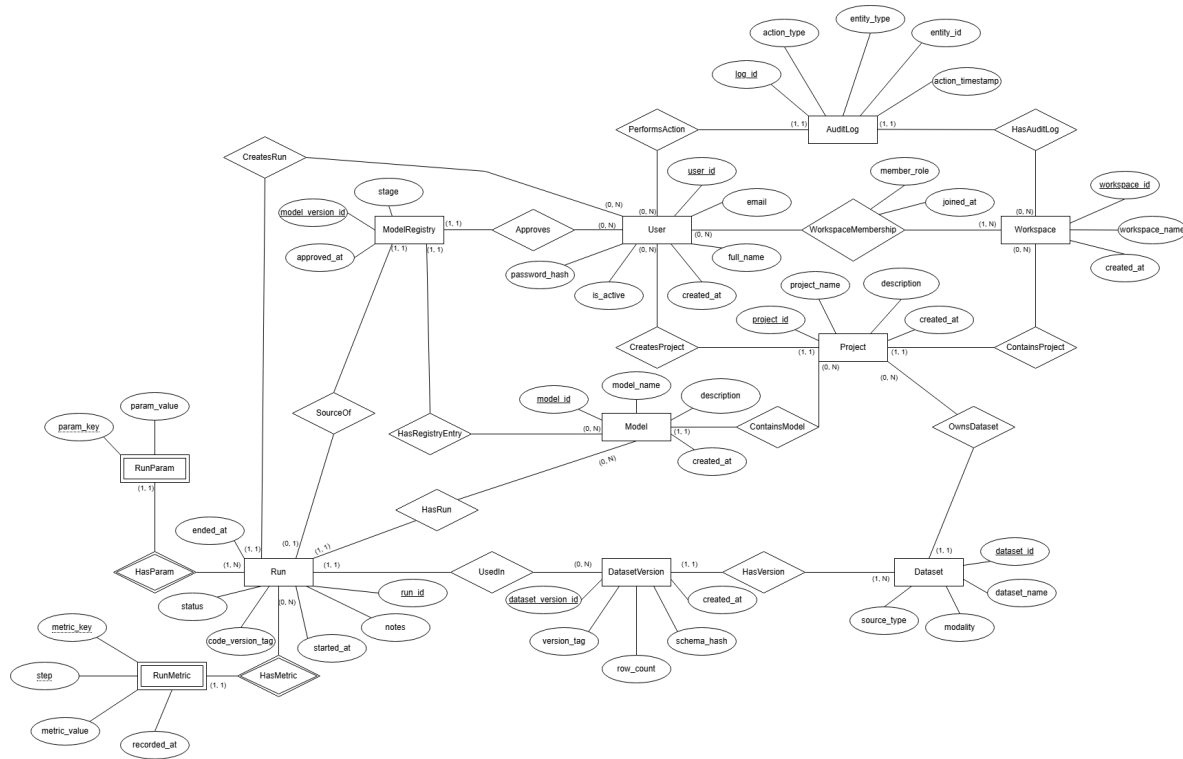
4. The interface must be implemented as a functional web application rather than static HTML only.
5. The database schema must reflect the ER design and relational mapping from earlier phases.
6. The schema must be normalized through Third Normal Form where applicable.
7. The system must use primary keys, foreign keys, and appropriate referential integrity constraints.
8. The project must include SQL scripts for creating and loading the database.
9. The SQL injection assignment must be demonstrated during the Phase 4 presentation.
10. The application must remain small enough to run locally within the class demo environment.

IV.6 Requirement-to-Implementation Summary

The implemented LabBench system satisfies the main project requirements by combining a normalized MySQL schema with a PHP web interface. Login and session management features support simultaneous access for multiple users. Workspace, project, dataset, model, run, parameter, metric, registry, and audit log tables support the core experiment-tracking workflow. The PHP pages provide the user interface for browsing, creating, updating, deleting, filtering, and comparing records. Referential integrity constraints protect the relationships between entities, while cascading rules apply to dependent run details such as parameters and metrics. The SQL injection demo satisfies the separate security assignment by showing both the vulnerable query behavior and the prepared-statement solution.

V. Logical Database Design

V.1 ER Diagram



V.2 Entity and Relationship Notes

- A workspace is the collaboration container for a team.
- A project belongs to one workspace and organizes related machine learning work.
- A model represents an experiment family within a project.
- A run is one training attempt of a model using a specific dataset version and parameter set.
- A model registry entry records an approved model version derived from a run.

V.3 Participation and (Min, Max) Constraints

The ER model includes participation and (min, max) constraints to define how entities interact. Most relationships follow a one-to-many structure, where a parent entity (such as Workspace, Project, or Model) can have multiple associated child entities, while each child belongs to exactly one parent. For example, a workspace can contain multiple projects (0, N), while each project must belong to exactly one workspace (1, 1). Similarly, a model can have multiple runs, but each run is associated with exactly one model. Some relationships enforce mandatory participation, such as DatasetVersion belonging to a Dataset (1, 1), while others allow optional participation, such as Users creating runs or projects (0, N). These constraints ensure data integrity and accurately represent real-world relationships within the system.

- **User — WorkspaceMembership — Workspace**

- User: (0, N)
- Workspace: (1, N)

- **Workspace — ContainsProject — Project**

- Workspace: (0, N)
- Project: (1, 1)

- **User — CreatesProject — Project**

- User: (0, N)
- Project: (1, 1)

- **Project — OwnsDataset — Dataset**

- Project: (0, N)
- Dataset: (1, 1)

- **Dataset — HasVersion — DatasetVersion**

- Dataset: (1, N)
- DatasetVersion: (1, 1)

- **Project — ContainsModel — Model**

- Project: (0, N)
- Model: (1, 1)

- **Model — HasRun — Run**

- Model: (0, N)
- Run: (1, 1)

- **DatasetVersion — UsedIn — Run**

- DatasetVersion: (0, N)
- Run: (1, 1)
- **User — CreatesRun — Run**
 - User: (0, N)
 - Run: (1, 1)
- **Run — HasParam — RunParam**
 - Run: (1, N)
 - RunParam: (1, 1)
- **Run — HasMetric — RunMetric**
 - Run: (0, N)
 - RunMetric: (1, 1)
- **Model — HasRegistryEntry — ModelRegistry**
 - Model: (0, N)
 - ModelRegistry: (1, 1)
- **Run — SourceOf — ModelRegistry**
 - Run: (0, 1)
 - ModelRegistry: (1, 1)
- **User — Approves — ModelRegistry**
 - User: (0, N)
 - ModelRegistry: (1, 1)
- **Workspace — HasAuditLog — AuditLog**
 - Workspace: (0, N)
 - AuditLog: (1, 1)
- **User — PerformsAction — AuditLog**
 - User: (0, N)
 - AuditLog: (1, 1)

V.4 Relational Database Schema

List of Relations

- **Users**

Users(user_id, full_name, email, password_hash, created_at, is_active)

PK: user_id

- **Workspaces**

Workspaces(workspace_id, workspace_name, created_at)

PK: workspace_id

- **WorkspaceMembers**

WorkspaceMembers(workspace_id, user_id, member_role, joined_at)

PK: (workspace_id, user_id)

FK: workspace_id → Workspaces(workspace_id)

FK: user_id → Users(user_id)

- **Projects**

Projects(project_id, workspace_id, project_name, description, created_by_user_id, created_at)

PK: project_id

FK: workspace_id → Workspaces(workspace_id)

FK: created_by_user_id → Users(user_id)

- **Datasets**

Datasets(dataset_id, project_id, dataset_name, modality, source_type)

PK: dataset_id

FK: project_id → Projects(project_id)

- **DatasetVersions**

DatasetVersions(dataset_version_id, dataset_id, version_tag, row_count, schema_hash, created_at)

PK: dataset_version_id

FK: dataset_id → Datasets(dataset_id)

- **Models**

Models(model_id, project_id, model_name, description, created_at)

PK: model_id

FK: project_id → Projects(project_id)

- **Runs**

Runs(run_id, model_id, dataset_version_id, created_by_user_id, started_at, ended_at, status, code_version_tag, notes)

PK: run_id

FK: model_id → Models(model_id)

FK: dataset_version_id → DatasetVersions(dataset_version_id)

FK: created_by_user_id → Users(user_id)

- **RunParams**

RunParams(run_id, param_key, param_value)

PK: (run_id, param_key)

FK: run_id → Runs(run_id)

- **RunMetrics**

RunMetrics(run_id, metric_key, metric_value, step, recorded_at)

PK: (run_id, metric_key, step)

FK: run_id → Runs(run_id)

- **ModelRegistry**

ModelRegistry(model_version_id, model_id, source_run_id, stage, approved_by_user_id, approved_at)

PK: model_version_id

FK: model_id → Models(model_id)

FK: source_run_id → Runs(run_id)

FK: approved_by_user_id → Users(user_id)

- **AuditLog**

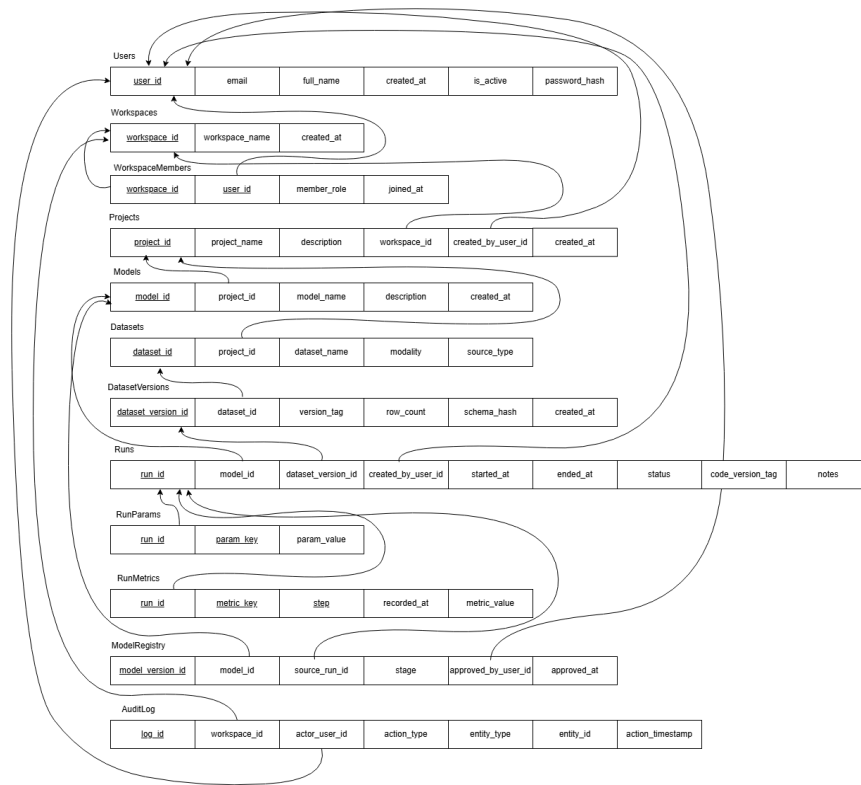
AuditLog(log_id, workspace_id, actor_user_id, action_type, entity_type, entity_id, action_timestamp)

PK: log_id

FK: workspace_id → Workspaces(workspace_id)

FK: actor_user_id → Users(user_id)

V.5 Relational Schema Diagram



V.6 Referential Integrity Constraints

Summary delete-action policy:

- WorkspaceMembers.workspace_id: CASCADE
- WorkspaceMembers.user_id: CASCADE
- RunParams.run_id: CASCADE
- RunMetrics.run_id: CASCADE
- All other foreign keys: RESTRICT

Cascade is used only for dependent detail records that have no meaning without their parent row. Restrict is used for major entities to preserve history, reproducibility, and accountability.

Detailed Overview

- WorkspaceMembers.workspace_id → Workspaces.workspace_id: CASCADE. If a workspace is deleted, its membership rows are deleted too.
- WorkspaceMembers.user_id → Users.user_id: CASCADE. If a user is deleted, that user's membership rows are deleted too.
- Projects.workspace_id → Workspaces.workspace_id: RESTRICT. A workspace cannot be deleted while projects still belong to it.
- Projects.created_by_user_id → Users.user_id: RESTRICT. A project must keep a valid creator reference.

- Datasets.project_id → Projects.project_id: RESTRICT. A project cannot be deleted while datasets still reference it.
- DatasetVersions.dataset_id → Datasets.dataset_id: RESTRICT. A dataset cannot be deleted while version rows still exist.
- Models.project_id → Projects.project_id: RESTRICT. A project cannot be deleted while models still belong to it.
- Runs.model_id → Models.model_id: RESTRICT. A model cannot be deleted while runs still reference it.
- Runs.dataset_version_id → DatasetVersions.dataset_version_id: RESTRICT. A dataset version cannot be deleted while runs still use it.
- Runs.created_by_user_id → Users.user_id: RESTRICT. A run must keep a valid creator reference.
- RunParams.run_id → Runs.run_id: CASCADE. If a run is deleted, its parameter rows are deleted too.
- RunMetrics.run_id → Runs.run_id: CASCADE. If a run is deleted, its metric rows are deleted too.
- ModelRegistry.model_id → Models.model_id: RESTRICT. A model cannot be deleted while registry entries still reference it.
- ModelRegistry.source_run_id → Runs.run_id: RESTRICT. A run cannot be deleted while a registry entry depends on it.
- ModelRegistry.approved_by_user_id → Users.user_id: RESTRICT. A registry entry must keep the approving user reference.
- AuditLog.workspace_id → Workspaces.workspace_id: RESTRICT. A workspace cannot be deleted while audit logs still reference it.
- AuditLog.actor_user_id → Users.user_id: RESTRICT. Audit records must retain the acting user.

V.7 Mapping from ER/EER Diagram to Relational Schema

Each strong entity in the ER design maps to its own relation. One-to-many relationships are represented by foreign keys on the many side (e.g., model_id in Runs, project_id in Models). The many-to-many relationship between Users and Workspaces maps to the associative relation WorkspaceMembers. RunParams and RunMetrics capture repeating run-level details as separate relations linked to Runs, using composite primary keys (run_id, param_key) and (run_id, metric_key, step) to uniquely identify entries. ModelRegistry records approved model versions and links each entry both to the parent model and to the source run from which the approved version was derived.

V.8 Normalization

V.8.1 Normalization Criteria Used

- 1NF: every attribute value is atomic; there are no repeating groups, multivalued attributes, or nested relations.

- 2NF: the relation is in 1NF and every non-prime attribute is fully functionally dependent on the whole candidate key; no partial dependency on a proper subset of a composite key is allowed.
- 3NF: the relation is in 2NF and for every non-trivial functional dependency $X \rightarrow A$, either X is a superkey or A is a prime attribute; equivalently, no non-key attribute is transitively dependent on a key through another non-key attribute.

V.8.2 Initial Schema to Final Schema

The initial logical schema carried forward from Phase 2 consisted of the same twelve relations analyzed in this document: Users, Workspaces, WorkspaceMembers, Projects, Datasets, DatasetVersions, Models, Runs, RunParams, RunMetrics, ModelRegistry, and AuditLog. After checking the schema against 1NF, 2NF, and 3NF, no decomposition of any relation was required.

Therefore, the final schema used for Phase 3 preserves the same relation structure as the initial schema. The only schema-level refinements made in Phase 3 were to make two alternate-key constraints explicit in the implemented design: UNIQUE(email) on Users and UNIQUE(source_run_id) on ModelRegistry. These changes strengthen the alignment between the logical design, the normalization analysis, and the implemented create.sql file, but they do not change the underlying set of relations.

V.8.3 Users

Schema	Users(user_id, full_name, email, password_hash, created_at, is_active)
Primary Key	user_id
Candidate Key(s)	user_id; email
Foreign Keys	None

Non-trivial functional dependencies:

- $\text{user_id} \rightarrow \text{full_name}, \text{email}, \text{password_hash}, \text{created_at}, \text{is_active}$
- $\text{email} \rightarrow \text{user_id}, \text{full_name}, \text{password_hash}, \text{created_at}, \text{is_active}$ (because email is declared UNIQUE)

1NF check: Yes. All attributes are atomic and each tuple represents one user account.

2NF check: Yes. The primary key is a single attribute, so partial dependencies are impossible.

3NF check: Yes. All non-key attributes depend directly on a candidate key, and declaring email UNIQUE simply adds an alternate key rather than creating a transitive dependency.

Note: Because email is declared UNIQUE in the revised Phase 3 schema, it functions as an alternate candidate key alongside user_id.

V.8.4 Workspaces

Schema	Workspaces(workspace_id, workspace_name, created_at)
Primary Key	workspace_id
Candidate Key(s)	workspace_id
Foreign Keys	None

Non-trivial functional dependencies:

- workspace_id → workspace_name, created_at

1NF check: Yes. All values are atomic and each tuple represents one workspace.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. No transitive dependency exists among non-key attributes.

V.8.5 WorkspaceMembers

Schema	WorkspaceMembers(workspace_id, user_id, member_role, joined_at)
Primary Key	(workspace_id, user_id)
Candidate Key(s)	(workspace_id, user_id)
Foreign Keys	workspace_id → Workspaces(workspace_id); user_id → Users(user_id)

Non-trivial functional dependencies:

- (workspace_id, user_id) → member_role, joined_at

1NF check: Yes. Each row stores one membership record and all values are atomic.

2NF check: Yes. member_role and joined_at depend on the whole composite key, not on workspace_id alone or user_id alone.

3NF check: Yes. There is no transitive dependency among the non-key attributes.

V.8.6 Projects

Schema	Projects(project_id, workspace_id, project_name, description, created_by_user_id, created_at)
Primary Key	project_id
Candidate Key(s)	project_id
Foreign Keys	workspace_id → Workspaces(workspace_id); created_by_user_id → Users(user_id)

Non-trivial functional dependencies:

- project_id → workspace_id, project_name, description, created_by_user_id, created_at

1NF check: Yes. All attributes are atomic for the purposes of the relational design.

2NF check: Yes. The primary key is single-attribute.

3NF check: Yes. Foreign keys do not introduce transitive dependencies within Projects because attributes from Workspaces and Users are not duplicated here.

V.8.7 Datasets

Schema	Datasets(dataset_id, project_id, dataset_name, modality, source_type)
Primary Key	dataset_id
Candidate Key(s)	dataset_id
Foreign Keys	project_id → Projects(project_id)

Non-trivial functional dependencies:

- dataset_id → project_id, dataset_name, modality, source_type

1NF check: Yes. All values are atomic and each tuple represents one dataset record.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. No non-key attribute is determined by another non-key attribute under the current design.

V.8.8 DatasetVersions

Schema	DatasetVersions(dataset_version_id, dataset_id, version_tag, row_count, schema_hash, created_at)
Primary Key	dataset_version_id
Candidate Key(s)	dataset_version_id
Foreign Keys	dataset_id → Datasets(dataset_id)

Non-trivial functional dependencies:

- dataset_version_id → dataset_id, version_tag, row_count, schema_hash, created_at

1NF check: Yes. All values are atomic.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. All non-key attributes depend directly on dataset_version_id and not on one another.

V.8.9 Models

Schema	Models(model_id, project_id, model_name, description, created_at)
Primary Key	model_id
Candidate Key(s)	model_id
Foreign Keys	project_id → Projects(project_id)

Non-trivial functional dependencies:

- $\text{model_id} \rightarrow \text{project_id}, \text{model_name}, \text{description}, \text{created_at}$

1NF check: Yes. Each tuple stores one model / experiment-family record with atomic attributes.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. No transitive dependency exists among non-key attributes.

V.8.10 Runs

Schema	Runs(run_id, model_id, dataset_version_id, created_by_user_id, started_at, ended_at, status, code_version_tag, notes)
Primary Key	run_id
Candidate Key(s)	run_id
Foreign Keys	model_id $\rightarrow$ Models(model_id); dataset_version_id $\rightarrow$ DatasetVersions(dataset_version_id); created by user id $\rightarrow$ Users(user_id)

Non-trivial functional dependencies:

- $\text{run_id} \rightarrow \text{model_id}, \text{dataset_version_id}, \text{created_by_user_id}, \text{started_at}, \text{ended_at}, \text{status}, \text{code_version_tag}, \text{notes}$

1NF check: Yes. Each tuple stores one run and all attributes are atomic.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. The relation stores only identifiers of related entities, not duplicated descriptive attributes from those entities.

V.8.11 RunParams

Schema	RunParams(run_id, param_key, param_value)
Primary Key	(run_id, param_key)
Candidate Key(s)	(run_id, param_key)
Foreign Keys	run_id $\rightarrow$ Runs(run_id)

Non-trivial functional dependencies:

- $(\text{run_id}, \text{param_key}) \rightarrow \text{param_value}$

1NF check: Yes. param_value is treated as one atomic value, even when it stores a numeric-looking value or a string.

2NF check: Yes. param_value depends on the whole composite key; it is not determined by run_id alone or param_key alone.

3NF check: Yes. There is no transitive dependency because the only non-key attribute is param_value.

V.8.12 RunMetrics

Schema	RunMetrics(run_id, metric_key, metric_value, step, recorded_at)
Primary Key	(run_id, metric_key, step)
Candidate Key(s)	(run_id, metric_key, step)
Foreign Keys	run_id → Runs(run_id)

Non-trivial functional dependencies:

- (run_id, metric_key, step) → metric_value, recorded_at

1NF check: Yes. Each tuple stores one atomic metric observation for one run, one metric name, and one step.

2NF check: Yes, under the stated interpretation that recorded_at belongs to the individual metric record. Then both metric_value and recorded_at depend on the whole composite key.

3NF check: Yes, under that same interpretation. No non-key attribute determines another non-key attribute.

V.8.13 ModelRegistry

Schema	ModelRegistry(model_version_id, model_id, source_run_id, stage, approved_by_user_id, approved_at)
Primary Key	model_version_id
Candidate Key(s)	model_version_id; source_run_id (by the stated logical rule that a run can source at most one registry entry)
Foreign Keys	model_id → Models(model_id); source_run_id → Runs(run_id); approved_by_user_id → Users(user_id)

Non-trivial functional dependencies:

- model_version_id → model_id, source_run_id, stage, approved_by_user_id, approved_at
- source_run_id → model_id, stage, approved_by_user_id, approved_at (because source_run_id is declared UNIQUE to enforce the rule that one run can source at most one registry entry)

1NF check: Yes. Each tuple stores one approved registry record and all attributes are atomic.

2NF check: Yes. The designated primary key is a single attribute.

3NF check: Yes. Because source_run_id is now explicitly declared UNIQUE in the revised schema, it is an alternate candidate key rather than merely an assumed one, so the relation satisfies 3NF cleanly.

V.8.14 AuditLog

Schema	AuditLog(log_id, workspace_id, actor_user_id, action_type, entity_type, entity_id, action_timestamp)
Primary Key	log_id

Candidate Key(s)	log_id
Foreign Keys	workspace_id → Workspaces(workspace_id); actor_user_id → Users(user_id)

Non-trivial functional dependencies:

- log_id → workspace_id, actor_user_id, action_type, entity_type, entity_id, action_timestamp

1NF check: Yes. Each tuple represents one logged action and all values are atomic.

2NF check: Yes. The primary key is a single attribute.

3NF check: Yes. No non-key attribute determines another non-key attribute within this relation.

VI. Database Implementation

VI.1 Overview

The LabBench database was implemented using MySQL. All tables were created using **CREATE TABLE** statements with appropriate primary key, foreign key, and constraint definitions to ensure data integrity and enforce relationships between entities. Data was populated using the **LOAD DATA LOCAL INFILE** command to import structured CSV files into each table for testing and demonstration purposes.

VI.2 create.sql

```

/*
  LabBench
  Database setup for the PHP/MySQL demo
  create.sql
*/

CREATE DATABASE IF NOT EXISTS LabBench;
USE LabBench;

CREATE TABLE Users (
  user_id INT NOT NULL AUTO_INCREMENT,
  full_name VARCHAR(100) NOT NULL,

```

```

email VARCHAR(255) NOT NULL,
password_hash VARCHAR(255) NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
PRIMARY KEY (user_id),
CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB;

```

```

CREATE TABLE Workspaces (
workspace_id INT NOT NULL AUTO_INCREMENT,
workspace_name VARCHAR(100) NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (workspace_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE WorkspaceMembers (
workspace_id INT NOT NULL,
user_id INT NOT NULL,
member_role VARCHAR(20) NOT NULL DEFAULT 'member',
joined_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (workspace_id, user_id),
CONSTRAINT fk_workspacemembers_workspace
FOREIGN KEY (workspace_id)
REFERENCES Workspaces(workspace_id)
ON DELETE CASCADE,
CONSTRAINT fk_workspacemembers_user
FOREIGN KEY (user_id)
REFERENCES Users(user_id)
ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE Projects (
project_id INT NOT NULL AUTO_INCREMENT,
workspace_id INT NOT NULL,
project_name VARCHAR(150) NOT NULL,
description TEXT NULL,
created_by_user_id INT NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (project_id),
CONSTRAINT fk_projects_workspace

```



```

        FOREIGN KEY (workspace_id)
        REFERENCES Workspaces(workspace_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_projects_created_by_user
        FOREIGN KEY (created_by_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE Datasets (
    dataset_id INT NOT NULL AUTO_INCREMENT,
    project_id INT NOT NULL,
    dataset_name VARCHAR(150) NOT NULL,
    modality VARCHAR(50) NOT NULL,
    source_type VARCHAR(50) NOT NULL,
    PRIMARY KEY (dataset_id),
    CONSTRAINT fk_datasets_project
        FOREIGN KEY (project_id)
        REFERENCES Projects(project_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE DatasetVersions (
    dataset_version_id INT NOT NULL AUTO_INCREMENT,
    dataset_id INT NOT NULL,
    version_tag VARCHAR(50) NOT NULL,
    row_count INT NOT NULL DEFAULT 0,
    schema_hash VARCHAR(64) NULL DEFAULT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (dataset_version_id),
    CONSTRAINT fk_datasetversions_dataset
        FOREIGN KEY (dataset_id)
        REFERENCES Datasets(dataset_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE Models (
    model_id INT NOT NULL AUTO_INCREMENT,
    project_id INT NOT NULL,
    model_name VARCHAR(150) NOT NULL,

```

```

description TEXT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (model_id),
CONSTRAINT fk_models_project
    FOREIGN KEY (project_id)
    REFERENCES Projects(project_id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE Runs (
    run_id INT NOT NULL AUTO_INCREMENT,
    model_id INT NOT NULL,
    dataset_version_id INT NOT NULL,
    created_by_user_id INT NOT NULL,
    started_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ended_at DATETIME NULL DEFAULT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'queued',
    code_version_tag VARCHAR(100) NULL DEFAULT NULL,
    notes TEXT NULL DEFAULT NULL,
    PRIMARY KEY (run_id),
    CONSTRAINT fk_runs_model
        FOREIGN KEY (model_id)
        REFERENCES Models(model_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_runs_dataset_version
        FOREIGN KEY (dataset_version_id)
        REFERENCES DatasetVersions(dataset_version_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_runs_created_by_user
        FOREIGN KEY (created_by_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE RunParams (
    run_id INT NOT NULL,
    param_key VARCHAR(100) NOT NULL,
    param_value VARCHAR(100) NOT NULL,
    PRIMARY KEY (run_id, param_key),
    CONSTRAINT fk_runparams_run

```

```

        FOREIGN KEY (run_id)
        REFERENCES Runs(run_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

```

```

CREATE TABLE RunMetrics (
    run_id INT NOT NULL,
    metric_key VARCHAR(50) NOT NULL,
    metric_value DECIMAL(10,4) NOT NULL,
    step INT NOT NULL,
    recorded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (run_id, metric_key, step),
    CONSTRAINT fk_runmetrics_run
        FOREIGN KEY (run_id)
        REFERENCES Runs(run_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

```

```

CREATE TABLE ModelRegistry (
    model_version_id INT NOT NULL AUTO_INCREMENT,
    model_id INT NOT NULL,
    source_run_id INT NOT NULL,
    stage VARCHAR(20) NOT NULL DEFAULT 'staging',
    approved_by_user_id INT NOT NULL,
    approved_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (model_version_id),
    CONSTRAINT uq_modelregistry_source_run UNIQUE (source_run_id),
    CONSTRAINT fk_modelregistry_model
        FOREIGN KEY (model_id)
        REFERENCES Models(model_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_modelregistry_source_run
        FOREIGN KEY (source_run_id)
        REFERENCES Runs(run_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_modelregistry_approved_by_user
        FOREIGN KEY (approved_by_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
    ) ENGINE=InnoDB;

```

```

CREATE TABLE AuditLog (
    log_id INT NOT NULL AUTO_INCREMENT,
    workspace_id INT NOT NULL,
    actor_user_id INT NOT NULL,
    action_type VARCHAR(50) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id INT NOT NULL,
    action_timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (log_id),
    CONSTRAINT fk_auditlog_workspace
        FOREIGN KEY (workspace_id)
        REFERENCES Workspaces(workspace_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_auditlog_actor_user
        FOREIGN KEY (actor_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

VI.3 load.sql

```

/*
    LabBench
    Database setup for the PHP/MySQL demo
    load.sql
*/

USE LabBench;

LOAD DATA LOCAL INFILE 'Users.csv'
INTO TABLE Users
FIELDS TERMINATED BY ','

```

```
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(full_name, email, password_hash, created_at, is_active);
```

```
LOAD DATA LOCAL INFILE 'Workspaces.csv'
INTO TABLE Workspaces
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_name, created_at);
```

```
LOAD DATA LOCAL INFILE 'WorkspaceMembers.csv'
INTO TABLE WorkspaceMembers
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_id, user_id, member_role, joined_at);
```

```
LOAD DATA LOCAL INFILE 'Projects.csv'
INTO TABLE Projects
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_id, project_name, description, created_by_user_id, created_at);
```

```
LOAD DATA LOCAL INFILE 'Datasets.csv'
INTO TABLE Datasets
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(project_id, dataset_name, modality, source_type);
```

```
LOAD DATA LOCAL INFILE 'DatasetVersions.csv'
INTO TABLE DatasetVersions
FIELDS TERMINATED BY ','
```

```
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(dataset_id, version_tag, row_count, schema_hash, created_at);
```

```
LOAD DATA LOCAL INFILE 'Models.csv'
INTO TABLE Models
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(project_id, model_name, description, created_at);
```

```
LOAD DATA LOCAL INFILE 'Runs.csv'
INTO TABLE Runs
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(model_id, dataset_version_id, created_by_user_id, started_at, ended_at, status,
code_version_tag, notes);
```

```
LOAD DATA LOCAL INFILE 'RunParams.csv'
INTO TABLE RunParams
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(run_id, param_key, param_value);
```

```
LOAD DATA LOCAL INFILE 'RunMetrics.csv'
INTO TABLE RunMetrics
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY ''''
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(run_id, metric_key, metric_value, step, recorded_at);
```

```
LOAD DATA LOCAL INFILE 'ModelRegistry.csv'
INTO TABLE ModelRegistry
```

```
FIELDS TERMINATED BY ','  
OPTIONALLY ENCLOSED BY ''''  
LINES TERMINATED BY '\r\n'  
IGNORE 1 ROWS  
(model_id, source_run_id, stage, approved_by_user_id, approved_at);  
  
LOAD DATA LOCAL INFILE 'AuditLog.csv'  
INTO TABLE AuditLog  
FIELDS TERMINATED BY ','  
OPTIONALLY ENCLOSED BY ''''  
LINES TERMINATED BY '\r\n'  
IGNORE 1 ROWS  
(workspace_id, actor_user_id, action_type, entity_type, entity_id, action_timestamp);
```

VII. Application and User Interfaces

VII.1 Overview

LabBench provides various interfaces that the user can interact with, such as logging in, accessing their projects and associated workspaces, and creating or modifying runs, models, and datasets for their projects. Specifically, the user interacts with HTML forms that send requests to PHP scripts. These scripts execute SQL queries on the MySQL database and return results to the user interfaces.

VII.2 Interfaces and User Interactions

The main entry point to LabBench is the login page. The purpose of this page is to authenticate registered app users and register new users. For login, the system requires the user's email and password. After the user enters their credentials and clicks "Log In," the app processes the credentials and checks whether they match those stored in the MySQL database. If authentication is successful, the user is redirected to the projects page. However, if the authentication process fails, the app displays an error message indicating that the login credentials were invalid. On the login page, a new user can create an account to access LabBench. The user must enter their full name, email address, and password. The user can also set the account status to active or inactive.

Selecting the account status to active during account registration allows the account to log in to the app. However, selecting the inactive account status creates an account but does not allow users to log in to LabBench with it. After completing all the fields, clicking "Create Account" saves the new user to the MySQL database and reloads the login page, prompting the user to sign in with their new credentials.

When the user logs in, they are immediately met with the projects page. On this page, the user can access other pages, such as "All Runs," "Datasets," and "Model Registry," via the vertical navbar on the left. Additionally, the user can create, edit, and delete projects, and filter projects by workspace to reduce time spent searching for a specific project. There is also an "Open Project" button for each project that redirects users to the project details page.

Note: The vertical navbar described above is present on every page while the user session is active. This means the navbar is present on all pages except the login page.

When the "Open Project" button on the projects page is pressed, the app redirects the user to the project details page for that project. On the project details page, the user can view the models, datasets, and runs associated with the project. Additionally, the user can edit the basic project details, log a new run, add a model, or add a dataset. From this page, the user can also modify and delete models and datasets using buttons. In the project's runs, the user can click the "View" button to open a run details page for a specific run.

When the user navigates to the "All Runs" page via the vertical navbar, they are presented with a comprehensive list of all runs across every workspace they belong to. At the top of the page, a filtering panel allows the user to narrow down the displayed runs by status, project, or a text-based search query. The status filter provides a dropdown with the options "queued," "running," "completed," and "failed," while the project filter lists all projects accessible to the logged-in user, organized by workspace. The search field allows the user to search across code version tags, run notes, and model names. After setting the desired filters, clicking "Apply Filters" reloads the page and displays only the runs that match the specified criteria. Below the filtering panel, the user will find a button labeled "+ Log Run," which redirects them to the log run page where a new run can be recorded. If a project filter is currently active, the project selection on the log run page will be pre-populated with that project. The main section of the page displays the filtered runs in a table, with each row showing the run ID, workspace, project, model, dataset version, status, code version tag, and start time. Each row also contains a "View" button that redirects the user to the run details page for that specific run. At the bottom of the "All Runs" page, there is a "Compare Two Runs" panel. This panel presents two dropdown menus, labeled "Run A" and "Run B," each populated with all runs accessible to the logged-in user. After selecting a run for each dropdown, clicking "Compare Runs" redirects the user to the

Compare Runs page, where the two selected runs are displayed side by side along with a metric comparison table.

When the user navigates to the "Datasets" page via the vertical navbar, they are presented with a list of all datasets across every workspace they belong to. Near the top of the page, a workspace filter lets the user narrow the displayed datasets to a specific workspace. This filter updates the list automatically upon selection without requiring the user to click a separate submit button. Below the workspace filter, two summary statistics are displayed: the total number of datasets and the total number of dataset versions visible under the current filter. These are followed by the main dataset table, which lists each dataset's name, associated project and workspace, modality, source type, and version count. Each dataset name in the table is a clickable link that redirects the user to the dataset detail page for that specific dataset. Additionally, each row contains three action buttons. The "Edit" button redirects the user to an edit page where the dataset's basic details can be modified. The "+ Version" button redirects the user to the add dataset version page with the relevant dataset pre-selected. The "Delete" button permanently deletes the dataset along with all of its versions and any associated runs, and prompts the user with a confirmation dialog before proceeding. At the top of the page, above the workspace filter, there is a "+ New Dataset" button that redirects the user to the create dataset page. On this page, the user selects a project, then fills in the dataset name, modality, and source type. The form also requires the user to provide an initial dataset version, including a version tag, a row count, and an optional schema hash. Clicking "Create Dataset" saves both the new dataset and its initial version to the database, then redirects the user to the associated project's detail page. To add a subsequent version to an existing dataset, the user can click the "+ Version" button from the datasets table. On the add dataset version page, the user selects the target dataset from a dropdown and provides a version tag, row count, and optional schema hash. Clicking "Add Version" saves the new version and redirects the user back to the associated project's detail page.

When the user navigates to the "Model Registry" page via the vertical navbar, they are presented with a list of all approved model versions across every workspace they belong to. Near the top of the page, a workspace filter allows the user to narrow the displayed entries down to a specific workspace and updates the list automatically upon selection. A summary statistic displaying the total number of registry entries visible under the current filter is shown below the workspace filter. The main section of the page displays all registry entries in a table, with each row showing the model version ID, model name, associated project and workspace, source run ID, deployment stage, the name of the user who approved the entry, and the approval timestamp. Each entry's stage is visually distinguished by a colored badge: "production" entries in green, "staging" entries in yellow, and "archived" entries in blue. Each row contains an "Edit" button that redirects the user to an edit page where the entry's details can be modified. Additionally, for users with an admin role in the relevant workspace, a "Delete" button is available, which permanently removes the registry entry. Non-admin users do not see the "Delete" button for entries in their workspaces. At the top of the page, above the workspace filter, there is an "+ Add Entry" button

that redirects the user to the add registry entry page. On this page, the user selects a model from a dropdown populated with all models accessible to them. Once a model is selected, the source run dropdown is automatically filtered to display only the runs associated with that model, showing each run's ID, status, and start time. The user then selects a source run and chooses a deployment stage of "staging," "production," or "archived." Clicking "Add Entry" saves the new registry entry, recording the logged-in user as the approver, and redirects the user back to the model registry page.

When the user navigates to the "Workspace Members" page via the vertical navbar, they are presented with a membership management interface for the workspaces they belong to. At the top of the page, a workspace selector dropdown allows the user to switch between their workspaces and updates the page automatically upon selection. The main section of the page displays a table listing all members of the selected workspace, with each row showing the member's full name, email address, role, and the date they joined. The behavior of this page depends on whether the logged-in user has an admin role in the selected workspace. For non-admin users, the members table is read-only, with each member's role displayed as plain text and no additional action buttons present. For admin users, the page exposes additional management capabilities throughout. In the members table, each member's role is rendered as a dropdown menu with the options "admin," "member," and "viewer," and changing the selection immediately submits the update without requiring a separate save button. Each member row also contains a "Remove" button that removes that member from the workspace after the user confirms the action in a dialog. An admin cannot remove themselves from the workspace, so the "Remove" button is not shown on the admin's own row. Similarly, an admin cannot demote themselves from the "admin" role if they are the only remaining admin in the workspace. Above the members table, admin users also see an "Add Member" panel containing a user dropdown and a role selector. The user dropdown is populated with all registered users who are not yet members of the selected workspace. After selecting a user and a role, clicking "Add Member" adds that user to the workspace. If all registered users are already members of the workspace, the panel displays a message indicating that no eligible users remain to be added, and the form is hidden accordingly.

When the user navigates to the "Audit Log" page via the vertical navbar, they are presented with a chronological record of all activity across the workspaces they belong to, with the most recent entries shown first and the display capped at 500 rows. At the top of the page, a filtering panel lets the user narrow the displayed log entries across several dimensions. The workspace filter restricts entries to a specific workspace, while the action filter limits results to a particular action type: "create," "update," "delete," or "approve." The entity type filter narrows results by the kind of entity that was acted upon, with options including "workspace," "project," "workspace_member," "run," "model," "dataset," and "model_registry." The user may also specify a date range using the "From date" and "To date" fields to limit entries to a particular time window. After configuring the desired filters, clicking "Apply Filters" reloads the page with

the matching results. A "Reset" button is also available to clear all active filters and return the log to its default unfiltered state. Below the filtering panel, the log entries are displayed in a table. Each row shows the timestamp of the action, the workspace where it occurred, the full name and user ID of the actor who performed it, the action type rendered as a badge, the type of entity affected, and the entity's ID. If no entries match the current filter criteria, the table displays a message indicating that no results were found.

VII.3 Database Connectivity

LabBench connects to a MySQL database using PHP's PDO extension, configured in db.php. The connection targets a local MySQL server and the LabBench database using the credentials defined at the top of that file. PDO is initialized with exception-based error reporting and associative array fetch mode. If the connection fails, the application halts immediately and displays the error message. The \$pdo connection object established in db.php is made available to all other scripts via require_once.

VIII. Results

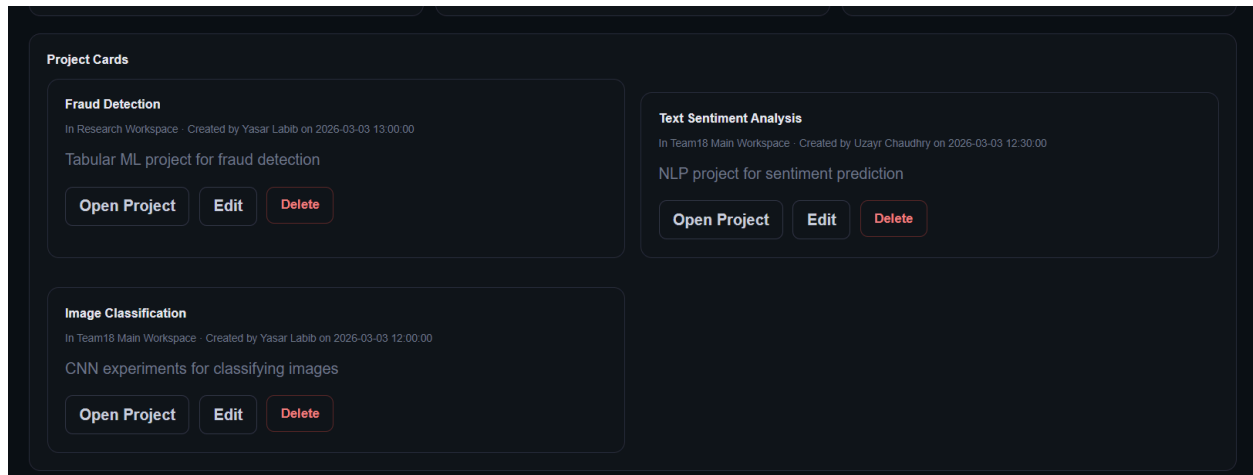
VIII.1 Sample Select Query

Purpose: This query retrieves the ID, name, description, and creation date of all projects belonging to workspaces that user ID 1 is a member of.

SQL Command:

```
SELECT p.project_id, p.project_name, p.description, p.created_at
FROM Projects p
INNER JOIN WorkspaceMembers wm ON wm.workspace_id = p.workspace_id
WHERE wm.user_id = 1;
```

Output:



VIII.2 Sample Update Query

Purpose: This query targets the project with ID 1 in the Projects table and updates its description column to the value "New description here," leaving all other columns and rows in the table unchanged.

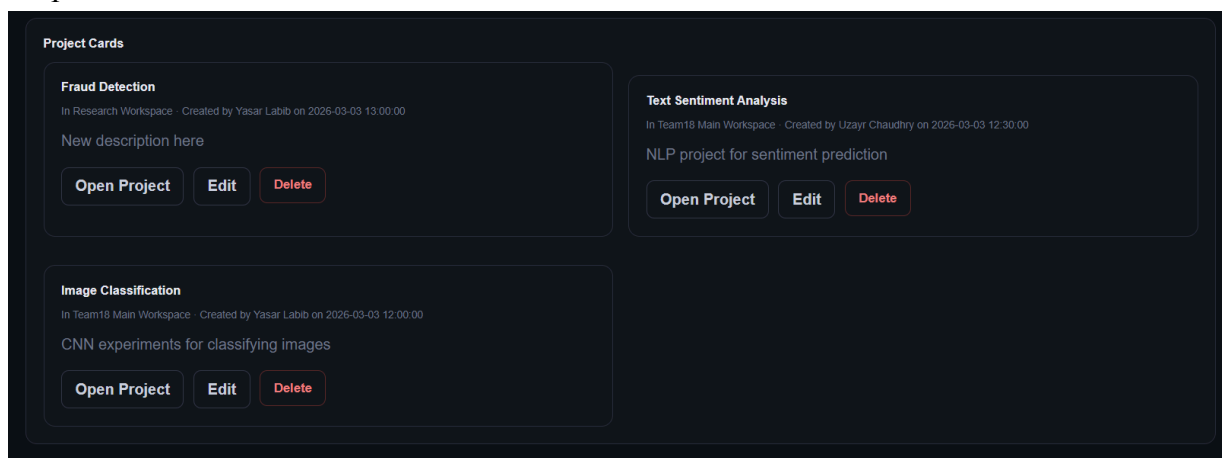
SQL Command:

UPDATE Projects

SET description = 'New description here'

WHERE project_id = 1;

Output:



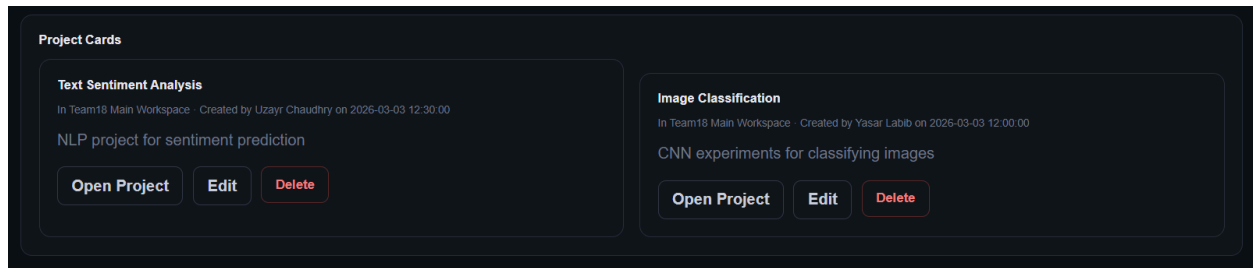
VIII.3 Sample Delete Query

Purpose: This query permanently removes the row corresponding to project ID 1 from the Projects table, deleting that project's record from the database entirely.

SQL Command:

```
DELETE FROM Projects WHERE project_id = 1;
```

Output:



VIII.4 Sample View Query

Purpose:

This view summarizes run information by joining Runs with Workspaces, Projects, Models, Datasets, DatasetVersions, and Users. It provides a readable report of each run and its experiment context.

SQL Command:

```
CREATE OR REPLACE VIEW RunSummaryView AS
```

```
SELECT
```

```
  r.run_id,  
  w.workspace_name,  
  p.project_name,  
  m.model_name,  
  d.dataset_name,  
  dv.version_tag AS dataset_version,  
  r.status,  
  r.code_version_tag,  
  r.started_at,  
  r.ended_at,  
  u.full_name AS created_by
```

```
FROM Runs r
```

```
JOIN Models m ON r.model_id = m.model_id
```

```
JOIN Projects p ON m.project_id = p.project_id
```

```
JOIN Workspaces w ON p.workspace_id = w.workspace_id
```

```
JOIN DatasetVersions dv ON r.dataset_version_id = dv.dataset_version_id
```

```
JOIN Datasets d ON dv.dataset_id = d.dataset_id
```

```
JOIN Users u ON r.created_by_user_id = u.user_id;
```

```
SELECT *
FROM RunSummaryView;
```

Output:

```
mysql> SELECT *
-> FROM RunSummaryView;
```

run_id	workspace_name	project_name	model_name	dataset_name	dataset_version	status	code_version_tag	started_at	ended_at	created_by
1	Team18 Main Workspace	Image Classification	ResNet Baseline	CIFAR-10	v1	completed	commit_a1	2026-03-06 08:00:00	2026-03-06 10:00:00	Yasar Labib
2	Team18 Main Workspace	Image Classification	ResNet Baseline	CIFAR-10	v2	completed	commit_b2	2026-03-06 11:00:00	2026-03-06 13:00:00	Uzayr Chaudhry
3	Team18 Main Workspace	Text Sentiment Analysis	BERT Sentiment	IMDB Reviews	v1	failed	commit_c3	2026-03-06 14:00:00	2026-03-06 16:00:00	Ugonna Anyalemechi
4	Research Workspace	Fraud Detection	XGBoost Fraud	Transaction Records	v1	completed	commit_d4	2026-03-07 09:00:00	2026-03-07 11:30:00	Yasar Labib

IX. Conclusion and Future Work

IX.1 Conclusion

LabBench successfully demonstrates the design and implementation of a relational database system for managing machine learning experiment metadata. The project addresses a real problem faced by machine learning students, researchers, and small engineering teams: experiment information is often stored across spreadsheets, notebooks, informal notes, or disconnected files, making it difficult to reproduce results, compare runs, track dataset versions, and maintain a reliable history of model development.

Through the previous project phases, the LabBench system was developed from a conceptual idea into a working database-backed web application. The project includes a complete relational schema, normalized tables, referential integrity constraints, SQL implementation, sample data loading, and a PHP/HTML/CSS user interface connected to a MySQL database. The final application supports the major workflow of:

Workspace > Project > Dataset/Model > Run > Parameters / Metrics > Model Registry > Audit Logs

The database design includes important entities such as Users, Workspaces, WorkspaceMembers, Projects, Datasets, DatasetVersions, Models, Runs, RunParams, RunMetrics, ModelRegistry, and AuditLog. These tables allow the system to store structured information about machine learning projects, dataset versions, model runs, hyperparameters, performance metrics, approved model versions, and user activity history.

A major strength of LabBench is that it uses a relational database instead of a flat spreadsheet-style design. This allows the system to enforce primary keys, foreign keys, many-to-one relationships, many-to-many membership relationships, and dependent detail tables. The schema also supports reproducibility by linking each run to a specific model, dataset

version, creator, code version, parameters, and metrics. This makes it possible for users to look back at previous experiments and understand exactly how a result was produced.

The implementation also demonstrates important database system concepts from the course. These include ER/EER modeling, relational mapping, functional dependency analysis, normalization up to Third Normal Form, SQL table creation, sample data loading, CRUD operations, query results, views, updates, deletes, and database connectivity through PHP. The application also includes login/session handling, protected pages, role-based access for admin areas, run comparison, audit logging, and a SQL injection demonstration showing both vulnerable query behavior and the safer prepared-statement solution.

Overall, LabBench meets the main goals of the project by showing how database design principles can be applied to a realistic machine learning experiment-tracking system. The final system is not only a database schema, but also a functional application that allows users to interact with the database through web pages and forms.

IX.2 Limitations

Although LabBench satisfies the main course project requirements, the system has some limitations due to the scope and time constraints of the semester.

First, LabBench is designed as a local course project demo rather than a production-ready application. The system runs in a local development environment and uses sample data for demonstration purposes. It does not include cloud deployment, production hosting, or full-scale multi-user infrastructure.

Second, the system stores machine learning experiment metadata, but it does not store raw datasets, trained model files, large artifacts, notebooks, or visualization files. This design keeps the database manageable, but it means the system would need additional file storage support in a real-world deployment.

Third, while the application includes authentication and session handling, security is still simplified for a class project. A production system would require stronger password handling, HTTPS, stricter access control, improved input validation, and more complete protection against common web vulnerabilities.

Fourth, the user interface is functional, but it is not as advanced as a commercial experiment-tracking platform. The current interface focuses on CRUD operations, viewing records, filtering, comparing runs, and demonstrating database connectivity. More advanced visual dashboards, charts, and interactive analytics could improve usability.

Finally, the system uses sample data and a limited number of workflows. A larger real-world system would need to support more complex experiment pipelines, larger teams, more detailed permissions, model artifact storage, deployment tracking, and integration with machine learning tools.

IX.3 Future Work

There are several ways LabBench could be improved in the future.

One possible enhancement is to add support for storing or linking external model artifacts, notebooks, logs, and dataset files. Instead of storing large files directly inside the database, LabBench could store file paths, cloud storage links, or artifact metadata. This would allow users to connect experiment metadata with the actual files used during training and evaluation.

Another improvement would be to add richer analytics and visualization features. For example, LabBench could include charts that compare run metrics over time, show accuracy/loss trends, rank models by performance, or visualize changes across dataset versions. This would make the system more useful for machine learning teams trying to evaluate model performance quickly.

A third improvement would be to strengthen user roles and permissions. The current system supports workspace membership and admin/member roles, but a more advanced version could include fine-grained permissions for creating projects, approving registry entries, deleting runs, editing datasets, and viewing audit logs.

LabBench could also be expanded with better search and filtering. Users could filter experiments by model type, dataset version, metric value, status, code version, date range, creator, or registry stage. This would make it easier to locate important experiments in larger projects.

Another future enhancement is to improve security. The SQL injection demonstration already shows the importance of prepared statements, but a production system would need stronger authentication, password hashing, CSRF protection, more robust validation, error handling, and secure deployment practices.

Finally, LabBench could be deployed as a real web application using a cloud-hosted database and web server. This would allow multiple users to access the system from different machines and collaborate on shared machine learning projects. With deployment, artifact support, improved security, and analytics, LabBench could become a more complete experiment-tracking platform.

X. References and Appendix

X.1 References

The following resources were used in the design, implementation, and preparation of the LabBench project:

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Course textbook and lecture material used for ER modeling, relational schema design, normalization, functional dependencies, and referential integrity.
2. CS 4347 Database Systems lecture slides and course materials. Used as guidance for database design, ER/EER diagrams, relational mapping, normalization, SQL implementation, and project requirements.
3. MySQL Documentation. Used as a reference for SQL syntax, table creation, primary keys, foreign keys, referential integrity constraints, views, inserts, updates, deletes, and data loading.
4. PHP Documentation. Used as a reference for server-side scripting, form handling, sessions, PDO database connectivity, and prepared statements.
5. HTML and CSS Documentation. Used as a reference for creating the web interface, forms, tables, navigation, and page styling.
6. LabBench project materials from previous phases, including the proposal, ER/EER design, relational schema, normalization work, SQL scripts, HTML/PHP interface files, sample data, and Phase 4 demo materials.

X.2 Appendix A: Full SQL Code

The complete SQL code for the project should be included in this appendix. This includes `create.sql` & `load.sql`.

The following SQL code was used to create and populate the LabBench database. The `create.sql` file defines the relational schema, primary keys, foreign keys, and referential integrity constraints. The `load.sql` file loads sample data into the database for testing and demonstration purposes.

create.sql

```
/*  
    LabBench  
    Phase 3; Task C  
    create.sql  
*/
```

```

CREATE DATABASE IF NOT EXISTS LabBench;
USE LabBench;

CREATE TABLE Users (
    user_id INT NOT NULL AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (user_id),
    CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB;

CREATE TABLE Workspaces (
    workspace_id INT NOT NULL AUTO_INCREMENT,
    workspace_name VARCHAR(100) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id)
) ENGINE=InnoDB;

CREATE TABLE WorkspaceMembers (
    workspace_id INT NOT NULL,
    user_id INT NOT NULL,
    member_role VARCHAR(20) NOT NULL DEFAULT 'member',
    joined_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id, user_id),
    CONSTRAINT fk_workspacemembers_workspace
        FOREIGN KEY (workspace_id)
        REFERENCES Workspaces(workspace_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_workspacemembers_user
        FOREIGN KEY (user_id)
        REFERENCES Users(user_id)
        ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Projects (
    project_id INT NOT NULL AUTO_INCREMENT,
    workspace_id INT NOT NULL,
    project_name VARCHAR(150) NOT NULL,
    description TEXT NULL,
    created_by_user_id INT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (project_id),
    CONSTRAINT fk_projects_workspace
        FOREIGN KEY (workspace_id)
        REFERENCES Workspaces(workspace_id)
        ON DELETE RESTRICT,

```

```

        CONSTRAINT fk_projects_created_by_user
            FOREIGN KEY (created_by_user_id)
            REFERENCES Users(user_id)
            ON DELETE RESTRICT
    ) ENGINE=InnoDB;

CREATE TABLE Datasets (
    dataset_id INT NOT NULL AUTO_INCREMENT,
    project_id INT NOT NULL,
    dataset_name VARCHAR(150) NOT NULL,
    modality VARCHAR(50) NOT NULL,
    source_type VARCHAR(50) NOT NULL,
    PRIMARY KEY (dataset_id),
    CONSTRAINT fk_datasets_project
        FOREIGN KEY (project_id)
        REFERENCES Projects(project_id)
        ON DELETE RESTRICT
    ) ENGINE=InnoDB;

CREATE TABLE DatasetVersions (
    dataset_version_id INT NOT NULL AUTO_INCREMENT,
    dataset_id INT NOT NULL,
    version_tag VARCHAR(50) NOT NULL,
    row_count INT NOT NULL DEFAULT 0,
    schema_hash VARCHAR(64) NULL DEFAULT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (dataset_version_id),
    CONSTRAINT fk_datasetversions_dataset
        FOREIGN KEY (dataset_id)
        REFERENCES Datasets(dataset_id)
        ON DELETE RESTRICT
    ) ENGINE=InnoDB;

CREATE TABLE Models (
    model_id INT NOT NULL AUTO_INCREMENT,
    project_id INT NOT NULL,
    model_name VARCHAR(150) NOT NULL,
    description TEXT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (model_id),
    CONSTRAINT fk_models_project
        FOREIGN KEY (project_id)
        REFERENCES Projects(project_id)
        ON DELETE RESTRICT
    ) ENGINE=InnoDB;

CREATE TABLE Runs (
    run_id INT NOT NULL AUTO_INCREMENT,
    model_id INT NOT NULL,
    dataset_version_id INT NOT NULL,

```

```

        created_by_user_id INT NOT NULL,
        started_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
        ended_at DATETIME NULL DEFAULT NULL,
        status VARCHAR(20) NOT NULL DEFAULT 'queued',
        code_version_tag VARCHAR(100) NULL DEFAULT NULL,
        notes TEXT NULL DEFAULT NULL,
        PRIMARY KEY (run_id),
        CONSTRAINT fk_runs_model
            FOREIGN KEY (model_id)
            REFERENCES Models(model_id)
            ON DELETE RESTRICT,
        CONSTRAINT fk_runs_dataset_version
            FOREIGN KEY (dataset_version_id)
            REFERENCES DatasetVersions(dataset_version_id)
            ON DELETE RESTRICT,
        CONSTRAINT fk_runs_created_by_user
            FOREIGN KEY (created_by_user_id)
            REFERENCES Users(user_id)
            ON DELETE RESTRICT
    ) ENGINE=InnoDB;

CREATE TABLE RunParams (
    run_id INT NOT NULL,
    param_key VARCHAR(100) NOT NULL,
    param_value VARCHAR(100) NOT NULL,
    PRIMARY KEY (run_id, param_key),
    CONSTRAINT fk_runparams_run
        FOREIGN KEY (run_id)
        REFERENCES Runs(run_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

CREATE TABLE RunMetrics (
    run_id INT NOT NULL,
    metric_key VARCHAR(50) NOT NULL,
    metric_value DECIMAL(10,4) NOT NULL,
    step INT NOT NULL,
    recorded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (run_id, metric_key, step),
    CONSTRAINT fk_runmetrics_run
        FOREIGN KEY (run_id)
        REFERENCES Runs(run_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

CREATE TABLE ModelRegistry (
    model_version_id INT NOT NULL AUTO_INCREMENT,
    model_id INT NOT NULL,
    source_run_id INT NOT NULL,
    stage VARCHAR(20) NOT NULL DEFAULT 'staging',

```

```

    approved_by_user_id INT NOT NULL,
    approved_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (model_version_id),
    CONSTRAINT uq_modelregistry_source_run UNIQUE (source_run_id),
    CONSTRAINT fk_modelregistry_model
        FOREIGN KEY (model_id)
        REFERENCES Models(model_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_modelregistry_source_run
        FOREIGN KEY (source_run_id)
        REFERENCES Runs(run_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_modelregistry_approved_by_user
        FOREIGN KEY (approved_by_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE AuditLog (
    log_id INT NOT NULL AUTO_INCREMENT,
    workspace_id INT NOT NULL,
    actor_user_id INT NOT NULL,
    action_type VARCHAR(50) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id INT NOT NULL,
    action_timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (log_id),
    CONSTRAINT fk_auditlog_workspace
        FOREIGN KEY (workspace_id)
        REFERENCES Workspaces(workspace_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_auditlog_actor_user
        FOREIGN KEY (actor_user_id)
        REFERENCES Users(user_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

load.sql

```
/*
    LabBench
    Phase 3; Task C
    load.sql
*/

USE LabBench;

LOAD DATA LOCAL INFILE 'Users.csv'
INTO TABLE Users
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(full_name, email, password_hash, created_at, is_active);

LOAD DATA LOCAL INFILE 'Workspaces.csv'
INTO TABLE Workspaces
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_name, created_at);

LOAD DATA LOCAL INFILE 'WorkspaceMembers.csv'
INTO TABLE WorkspaceMembers
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_id, user_id, member_role, joined_at);

LOAD DATA LOCAL INFILE 'Projects.csv'
INTO TABLE Projects
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_id, project_name, description, created_by_user_id, created_at);

LOAD DATA LOCAL INFILE 'Datasets.csv'
INTO TABLE Datasets
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(project_id, dataset_name, modality, source_type);
```

```

LOAD DATA LOCAL INFILE 'DatasetVersions.csv'
INTO TABLE DatasetVersions
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(dataset_id, version_tag, row_count, schema_hash, created_at);

LOAD DATA LOCAL INFILE 'Models.csv'
INTO TABLE Models
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(project_id, model_name, description, created_at);

LOAD DATA LOCAL INFILE 'Runs.csv'
INTO TABLE Runs
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(model_id, dataset_version_id, created_by_user_id, started_at, ended_at,
status, code_version_tag, notes);

LOAD DATA LOCAL INFILE 'RunParams.csv'
INTO TABLE RunParams
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(run_id, param_key, param_value);

LOAD DATA LOCAL INFILE 'RunMetrics.csv'
INTO TABLE RunMetrics
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(run_id, metric_key, metric_value, step, recorded_at);

LOAD DATA LOCAL INFILE 'ModelRegistry.csv'
INTO TABLE ModelRegistry
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(model_id, source_run_id, stage, approved_by_user_id, approved_at);

LOAD DATA LOCAL INFILE 'AuditLog.csv'

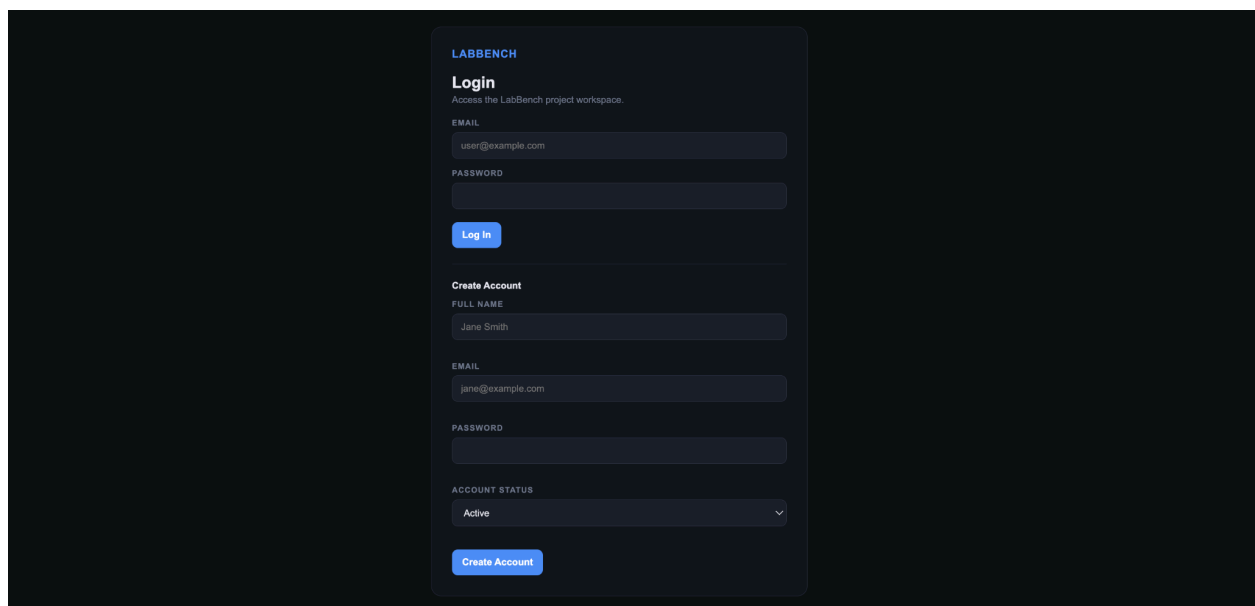
```

```
INTO TABLE AuditLog
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(workspace_id, actor_user_id, action_type, entity_type, entity_id,
action_timestamp);
```

X.3 Appendix B: Interface Screenshots

The following screenshots show the main user interfaces of the LabBench application. These pages allow users to log in, navigate through workspaces and projects, manage datasets, models, runs, parameters, metrics, registry entries, and audit logs, and demonstrate SQL injection behavior and prevention.

1. Login page



2. Workspaces page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Workspaces

Signed in as Yasar Labib

Workspace / Management

Workspaces

Create a workspace, view the ones you belong to, and delete the ones you administer.

MY WORKSPACES

2

ADMIN OF

2

MEMBERS ACROSS THEM

6

Create Workspace

WORKSPACE NAME

Create Workspace

Your Workspaces

NAME	YOUR ROLE	MEMBERS	PROJECTS	CREATED	
Research Workspace	admin	2	1	2024-03-02 10:30:00	Delete
Team18 Main Workspace	admin	4	2	2024-03-02 10:00:00	Delete

3. Workspace Members page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Workspace Members

Signed in as Yasar Labib

Workspace Members

View and manage membership for workspaces you belong to.

Workspace

SELECT WORKSPACE

Research Workspace

Add Member

USER

Uzayr Chaudhry — uzayr@labbench.com

ROLE

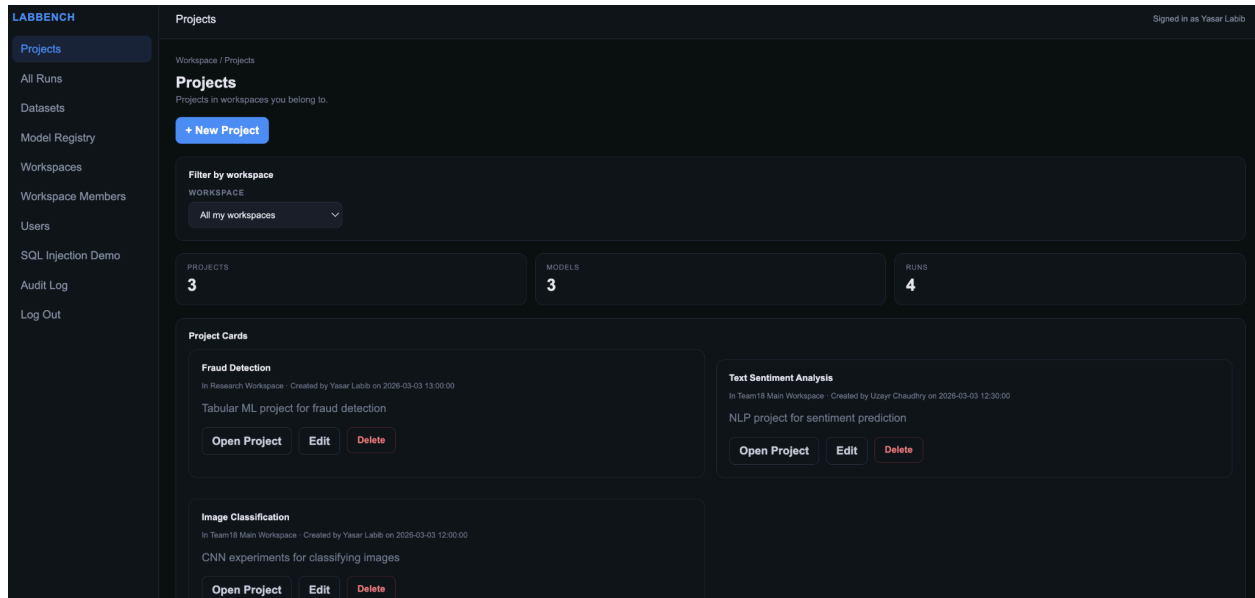
member

Add Member

Members

NAME	EMAIL	ROLE	JOINED	
Ugonna Anyalemechi	ugonna@labbench.com	member	2024-03-02 11:25:00	Remove
Yasar Labib	yasar@labbench.com	admin	2024-03-02 11:20:00	—

4. Projects page



The screenshot shows the LABBENCH interface with the 'Projects' page selected in the sidebar. The main content area is titled 'Projects' and shows a list of projects in a workspace. A '+ New Project' button is visible. Below the button, there's a 'Filter by workspace' section with a dropdown menu set to 'All my workspaces'. A summary bar shows 3 Projects, 3 Models, and 4 Runs. The main area displays three project cards: 'Fraud Detection', 'Text Sentiment Analysis', and 'Image Classification'. Each card includes a description, creation date, and buttons for 'Open Project', 'Edit', and 'Delete'.

LABBENCH

Projects

Signed in as Yasar Labib

Workspace / Projects

Projects

Projects in workspaces you belong to.

[+ New Project](#)

Filter by workspace

WORKSPACE

All my workspaces

PROJECTS 3

MODELS 3

RUNS 4

Project Cards

Fraud Detection

In Research Workspace · Created by Yasar Labib on 2026-03-03 13:00:00

Tabular ML project for fraud detection

[Open Project](#) [Edit](#) [Delete](#)

Text Sentiment Analysis

In Team18 Main Workspace · Created by Uzayr Chaudhry on 2026-03-03 12:30:00

NLP project for sentiment prediction

[Open Project](#) [Edit](#) [Delete](#)

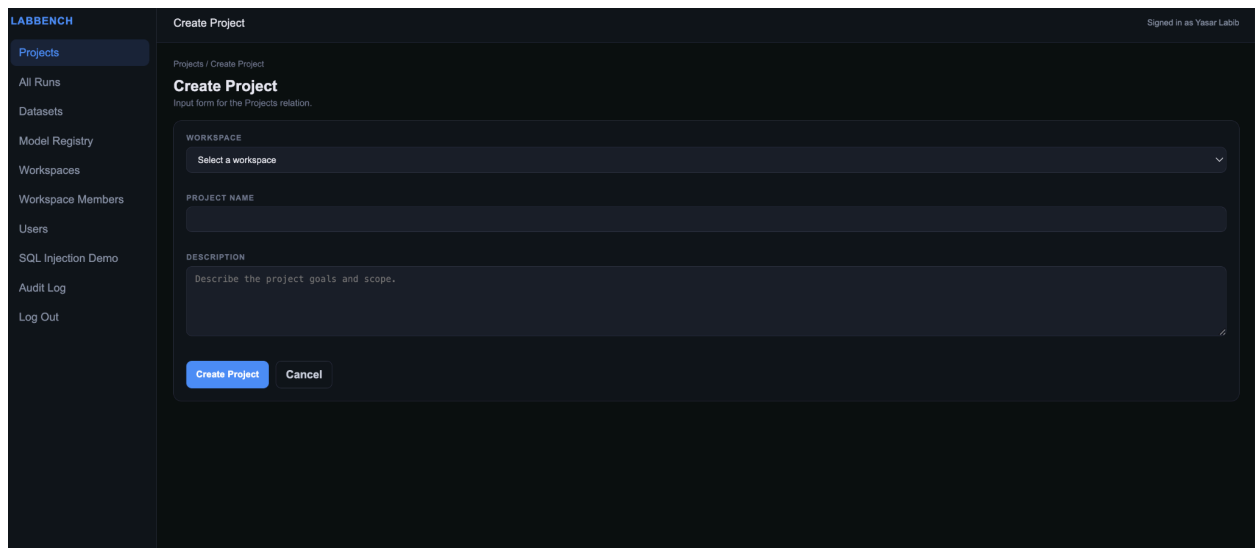
Image Classification

In Team18 Main Workspace · Created by Yasar Labib on 2026-03-03 12:00:00

CNN experiments for classifying images

[Open Project](#) [Edit](#) [Delete](#)

5. Create project form



The screenshot shows the LABBENCH interface with the 'Create Project' form. The sidebar is on the left, and the main content area is titled 'Create Project'. The form has three sections: 'WORKSPACE' with a dropdown menu, 'PROJECT NAME' with a text input field, and 'DESCRIPTION' with a text area. At the bottom, there are 'Create Project' and 'Cancel' buttons.

LABBENCH

Projects

Signed in as Yasar Labib

Create Project

Input form for the Projects relation.

WORKSPACE

Select a workspace

PROJECT NAME

DESCRIPTION

Describe the project goals and scope.

[Create Project](#) [Cancel](#)

6. Dataset page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Datasets

Signed in as Yasar Labib

Workspace / Datasets

Datasets

Datasets across workspaces you belong to.

+ New Dataset

Filter by workspace

WORKSPACE

All my workspaces

DATASETS

3

VERSIONS

4

Dataset Registry

DATASET NAME	PROJECT	MODALITY	SOURCE	VERSIONS	
Transaction Records	Fraud Detection — Research Workspace	TABULAR	internal	1	Edit+ VersionDelete
IMDB Reviews	Text Sentiment Analysis — Team18 Main Workspace	TEXT	public	1	Edit+ VersionDelete
CIFAR-10	Image Classification — Team18 Main Workspace	IMAGE	public	2	Edit+ VersionDelete

7. Create dataset page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Create Dataset

Signed in as Yasar Labib

Datasets / Create Dataset

Create Dataset

Input form for Datasets and the initial DatasetVersions entry.

PROJECT

Select a project

Dataset

DATASET NAME

MODALITY

Select modality

SOURCE TYPE

Select source type

Initial Version

VERSION TAG

v1.0

ROW COUNT

0

SCHEMA HASH (OPTIONAL)

Create Dataset

Cancel

51

8. Model Registry page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Model Registry

Signed in as Yasar Labib

Workspace / Model Registry

Model Registry

Approved model versions across your workspaces.

+ Add Entry

Filter by workspace

WORKSPACE

All my workspaces

REGISTRY ENTRIES

2

Registry Entries

MODEL VERSION ID	MODEL	PROJECT	SOURCE RUN	STAGE	APPROVED BY	APPROVED AT	
2	XGBoost Fraud	Fraud Detection — Research Workspace	4	PRODUCTION	Yasar Labib	2026-03-08 11:00:00	EditDelete
1	ResNet Baseline	Image Classification — Team18 Main Workspace	2	STAGING	Yasar Labib	2026-03-08 10:00:00	EditDelete

9. Runs page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

All Runs

Signed in as Yasar Labib

Workspace / Runs

All Runs

Browse, filter, and compare runs in workspaces you belong to.

Run Filters

STATUS

All Statuses

PROJECT

All Projects

SEARCH

Search code tag, notes, or model

Apply Filters

+ Log Run

Run Results

RUN ID	WORKSPACE	PROJECT	MODEL	DATASET VERSION	STATUS	CODE TAG	STARTED	
4	Research Workspace	Fraud Detection	XGBoost Fraud	v1	COMPLETED	commit_d4	2026-03-07 09:00:00	View
3	Team18 Main Workspace	Text Sentiment Analysis	BERT Sentiment	v1	FAILED	commit_d3	2026-03-06 14:00:00	View
2	Team18 Main Workspace	Image Classification	ResNet Baseline	v2	COMPLETED	commit_b2	2026-03-06 11:00:00	View
1	Team18 Main Workspace	Image Classification	ResNet Baseline	v1	COMPLETED	commit_a1	2026-03-06 08:00:00	View

Compare Two Runs

RUN A

Select first run

RUN B

Select second run

Compare Runs

10. Log run form

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Log Run

Runs / Log Run

Log Run

Create a run with an optional initial parameter and optional initial metric.

PROJECT FILTER

All Projects

MODEL

Select a model

DATASET VERSION

Select a dataset version

STATUS

queued

STARTED AT

mm/dd/yyyy, --:-- --

ENDED AT

mm/dd/yyyy, --:-- --

CODE VERSION TAG

NOTES

Initial Hyperparameter (Optional)

PARAMETER KEY

learning_rate

PARAMETER VALUE

0.001

Initial Metric (Optional)

METRIC KEY

accuracy

METRIC VALUE

0.91

STEP

1

RECORDED AT

mm/dd/yyyy, --:-- --

Save Run

Cancel

11. Run comparison page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Compare Runs

Runs / Compare Runs

Compare Runs

Compare two selected runs side by side.

RUN A

Run 4 — Fraud Detection — XGBoost Fraud — completed

RUN B

Run 3 — Text Sentiment Analysis — BERT Sentiment — failed

Refresh Comparison

Run A

RUN ID	4
WORKSPACE	Research Workspace
PROJECT	Fraud Detection
MODEL	XGBoost Fraud
DATASET VERSION	v1
STATUS	COMPLETED
CODE VERSION	commit_d4

Run B

RUN ID	3
WORKSPACE	Team10 Main Workspace
PROJECT	Text Sentiment Analysis
MODEL	BERT Sentiment
DATASET VERSION	v1
STATUS	FAILED
CODE VERSION	commit_c3

Metric Comparison

METRIC	STEP	RUN A	RUN B
accuracy	1		0.1010
f1	1	0.9123	
precision	1	0.9050	

12. Audit log page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

Audit Log

Showing up to 500 rows · newest first

Audit Log

Activity in workspaces you belong to.

Filters

WORKSPACE

All

ACTION

All

ENTITY TYPE

All

FROM DATE

mm/dd/yyyy

TO DATE

mm/dd/yyyy

Apply filters

Reset

Entries (9)

TIME	WORKSPACE	ACTOR	ACTION	ENTITY	ID
2026-03-08 11:00:00	Research Workspace	Yasar Labib (#7)	APPROVE	model_registry	2
2026-03-08 10:00:00	Team18 Main Workspace	Yasar Labib (#7)	APPROVE	model_registry	1
2026-03-03 13:00:00	Research Workspace	Yasar Labib (#7)	CREATE	project	3
2026-03-03 12:30:00	Team18 Main Workspace	Uday Chaudhry (#2)	CREATE	project	2
2026-03-03 12:00:00	Team18 Main Workspace	Yasar Labib (#7)	CREATE	project	1

13. SQL injection vulnerable demo page

LABBENCH

Projects

All Runs

Datasets

Model Registry

Workspaces

Workspace Members

Users

SQL Injection Demo

Audit Log

Log Out

SQL Injection Demo

Signed in as Yasar Labib

Workspace / SQL Injection Demo

SQL Injection Assignment

Demonstrates the unsafe SELECT routine and the prepared-statement rewrite using the same LabBench run-search form.

Demo script for the TA

Use the same input twice. First choose Vulnerable SELECT, then choose Prepared Statement SELECT. For the injection test, put this exact value in Code Version Tag and leave the other fields blank:

not-real' OR '1'='1

The vulnerable version returns rows even though the code tag is not accurate. The prepared-statement version treats the payload as text and should return zero rows.

Run Search Form

ROUTINE

Select routine

PROJECT NAME

Image Classification

MODEL NAME

ResNet Baseline

RUN STATUS

completed

CODE VERSION TAG

commit_a1

Run SELECT

Reset

54

14. SQL injection prepared-statement demo page

Run Search Form

ROUTINE

Part A: Vulnerable SELECT

PROJECT NAME

Image Classification

MODEL NAME

ResNet Baseline

RUN STATUS

completed

CODE VERSION TAG

commit_e1

Run SELECT

Reset

Part A: Generated Vulnerable SQL

```
SELECT
  r.run_id,
  w.workspace_name,
  p.project_name,
  m.model_name,
  dv.version_tag,
  r.status,
  r.code_version_tag,
  r.created_by,
  r.created_at
FROM Run r
INNER JOIN Models m ON m.model_id = r.model_id
INNER JOIN Projects p ON p.project_id = r.project_id
INNER JOIN Workspaces w ON w.workspace_id = r.workspace_id
INNER JOIN WorkspaceMembers wm ON wm.workspace_id = p.workspace_id
INNER JOIN DatasetVersions dv ON dv.dataset_version_id = r.dataset_version_id
INNER JOIN Users u ON u.user_id = r.created_by_user_id

WHERE wm.user_id = 1
ORDER BY r.run_id DESC
LIMIT 100
```

Query Results (4)

RUN ID	WORKSPACE	PROJECT	MODEL	DATASET VERSION	STATUS	CODE TAG	CREATED BY	STARTED
4	Research Workspace	Fraud Detection	XGBoost Fraud	v1	COMPLETED	commit_d4	Yasar Labb	2024-03-07 09:00:00
3	Team18 Main Workspace	Text Sentiment Analysis	BERT Sentiment	v1	FAILED	commit_c3	Ugonna Anyalemechi	2024-03-06 14:00:00
2	Team18 Main Workspace	Image Classification	ResNet Baseline	v2	COMPLETED	commit_b2	Uzayr Chaudhry	2024-03-06 11:00:00
1	Team18 Main Workspace	Image Classification	ResNet Baseline	v1	COMPLETED	commit_a1	Yasar Labb	2024-03-06 08:00:00